



LUNDS TEKNISKA
HÖGSKOLA
Lunds universitet

**Institutionen för
Informationsteknologi**

Projektrapport

Datoriserad Nummerpresentatör

VT 2004

Fredrik Turnerot, e00ft@efd.lth.se

Per Fridén, e00pf@efd.lth.se

Abstract

This document describes a computerized caller Identification system. The CIS is designed to decode incoming phone calls and present the callers telephone number on a computer. The computer is connected via a serial communication port to the hardware. Every valid decoded phone call is stored in the hardware together with the corresponding date and time. When a call is detected the telephone number is sent to the computer. A small red led shows if the hardware has detected any missing phone calls. Through a PC-program the list of telephone numbers and dates can be requested. Up to one thousand numbers can be stored in the hardware. If a person wishes it is possible to block specific numbers from reaching the telephone. The graphical user interface is written in Visual C++. The program is the strength of the system, which offers many different services. One special feature is automatic number search at Eniro's white pages. The program receives an unknown telephone number and looks it up at the white pages and presents the callers identity.

Innehållsförteckning

1. Inledning.....	3
2. Kravspecifikation	4
3. Konstruktion.....	5
4. Hårdvara	6
4.1. Processor, Motorola MC68008	6
4.2. 16 kb EPROM, Fairchild NM27C128	6
4.3. 32 kb SRAM, NEC μ PD43256B.....	6
4.4. Realtidsklocka, National Semiconductor MM 58274 C	7
4.5. DTMF-Transceiver, MITEL MT8880C	7
4.6. RS-232 Driver, Maxim MAX232	7
4.7. Kommunikationskrets, UART National PC16550D.....	7
4.8. Logikrets, Lattice 1016e	8
4.9. Svartlistning	8
4.10. Missat samtal	8
5. Mjukvara	9
5.1. Processor	9
5.2. Logik	10
5.3. PC-program	10
6. Resultat och Diskussion	11
7. Bilagor	12
7.1. Bilaga 1, Kopplingsschema	12
7.2. Bilaga 2, Processorprogram	13
7.3. Bilaga 3, Logikprogram	20

1. Inledning

Under läsperiod 1 våren 2004 läste vi kursen Digitala Projekt som ges vid Institutionen för Informationsteknologi vid Lunds Tekniska Högskola. Kursen gick ut på att implementera en idé i hårdvara och alltså nyttja sina teoretiska kunskaper på något verkligt.

Efter en del övervägande bestämde vi oss för att konstruera en nummerpresentatör med en del speciella funktioner som presentatörer på marknaden i allmänhet saknar. Vi ville bland annat att vår nummerpresentatör skulle kunna kommunicera och presentera inkomna nummer på en PC och att den skulle kunna blockera ej välkomna från att nå mottagaren (svartlistning). Fler funktioner finns beskrivna i kravspecifikationen.

Rent allmänt så är nummerpresentation en tjänst som Telia erbjuder vilken innebär att numret till den som ringer skickas till mottagaren i början av ringningen. Detta nummer kan avkodas och presenteras, vilket är grunden i det här projektet.

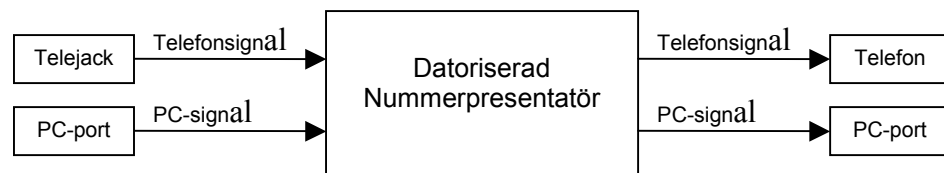
För att bygga en nummerpresentation enligt våra önskemål behövde vi först sätta oss in i hur nummerpresentation fungerar, bestämma hur konstruktionen skulle se ut, vilka kretsar som skulle användas samt bygga ihop allting och få det att fungera tillsammans. Vi hade sju veckor på oss från idé till fungerande prototyp, vilket blev sju intensiva men roliga veckor.

2. Kravspecifikation

Det första arbetet som inleddes efter att vi bestämt oss för att bygga en nummerpresentatör var att ta fram en kravspecifikation. Vid tidpunkten då specifikationen skrevs var vi inte insatta i vilka kretsar som fanns att tillgå eller andra tekniska detaljer vilket innebar en ganska allmän specifikation. Det var ändå viktigt att få ner våra idéer på papper så att vi hade något att utgå ifrån i vårt vidare arbete.

De krav vi satte upp var som följer:

Den datoriserade nummerpresentatören ska kunna lagra inkomna nummer och skicka över dessa till en PC för presentation.



Nummerpresentatören ska uppfylla följande:

- lagra 100 senaste inkomna nummer med tillhörande datum och tidpunkt
- överföra lagrade nummer till en PC
- överföra aktuellt nummer vid inkommande telefonsamtal
- presentera inkomna nummer med tillhörande data (namn m.m.)
- ignorera inkommande telefonsamtal för svartlistade nummer
- ta emot lista av svartlistade nummer (max 100 st.) från en PC
- presentera och editera svartlistade nummer
- ringa upp nummer från PC då telefonluren är lyft

Allt eftersom vi implementerade de olika funktionerna ändrade vi på små detaljer men samtliga punkter utgör grunden för vår konstruktion.

3. Konstruktion

I inledningen av konstruktionsarbetet var det viktigt att snabbt få fram ett detaljerat kretsschema för den datoriserade nummerpresentatören. Det kändes viktigt att veta vart projektet var på väg innan konstruktionsarbetet inleddes. När väl den funktionella delen av produkten var bestämd så gick det relativt lätt att välja ut de rätta kretsarna. Efter många timmars studie av datablad ritades slutligen ett kopplingsschema i Power Logic. I schemat inkluderades alla signaler mellan de olika kretsarna. När väl kopplingsschemat var färdigställt började planeringen av konstruktionsarbetet. Kopplingsschemat finns bifogat som bilaga 1.

Tanken bakom utvecklingen av konstruktionen var att ta små steg framåt och varje gång verifiera att resultatet blev korrekt. Den datoriserade nummerpresentatören bestod av två separata delar. En hårdvaronära del som skötte avkodning av signaler från telefonlinjen och en mjukvarobaserad del som presenterade resultat på en PC. Hårdvarodelen är konstruerad på ett klassiskt kopplingskort där kretsar och dylikt sammankopplas med manuellt dragna ledningar. Onödiga felkopplingar undveks genom att ständigt dubbelkolla allt som kopplades. Av alla fel och buggar som uppstått under utvecklingen berodde inga på feldragna ledningar. De mest centrala kretsarna som utgjorde grundidén för projektet var dels en avkodare av DTMF-signaler och en krets för seriell kommunikation. Det första delmålet blev att få fram en fungerande kommunikation mellan hårdvaran och PCn. Kommunikationen var central för hela produkten, utan den gick det inte att presentera data grafiskt. Det enda sättet att felsöka utan den seriella kommunikationen var att helt enkelt mäta och för hand hålla koll på alla logiska ettor och nollor. Med anslutningen till datorn kunde sådan information i viss mån presenteras på datorskärmen. För en fungerande kommunikation krävdes det att koppla samman processor, EPROM, SRAM, UART (kommunikationskrets) och styrlogik. Ett utvecklingsbord användes för att simulera funktionen av processorn och EPROMet. Utvecklingsbordet gjorde det lätt att programmera om processorns beteende och gjorde det dessutom möjligt att, via det grafiska interfacet, läsa och skriva till specifika delar av minnet. För verifiering av kommunikationen mellan datorn och hårdvaran användes en enkel windows-terminal. Data hårdkodades i programmet till processorn och skickades sedan över till PC för kontroll. När väl kommunikationen var stabil kunde den användas som stöd för verifieringen av de övriga kretsarna, i det här fallet realtidsklockan och DTMF-avkodaren.

4. Hårdvara

4.1. Processor, Motorola MC68008

Processorn har en 8 bitars databuss och 20 bitars adressering. De fyra mest signifikanta bitarna användes för styrning av vilka kretsar som skall adresseras, nedan följer en beskrivning av dessa.

Krets	Fyra översta adressben	Adressspann (Hex)
EPROM	0000	00000 – 03FFF
SRAM	0001	10000 – 17FFF
REAL_CLOCK	0010	20000 – 2000F
UART	0011	30000 – 3000F
DTMF R/W	0100	40000
DTMF CONTROL	0101	50000
Koppla in samtal	0110	60000
Tänd lysdiod	0111	70000
Släck lysdiod	1000	80000

Styrbitarna kopplades till styrlogiken vilken avgjorde vilken krets som skulle vara aktiv. Att läsa och skriva med processorn gick till på följande sätt. Först instruerades processorn att skriva på en specifik adress. Data, adress och R/W-signalen (indikation för läsning och skrivning) läggs ut på respektive utgång på processorn. Efter det att signalerna lagts ut tar det ett par klockpulser innan de blir stabila. Signalerna DS respektive AS (Data och Adress Stabil) genereras av processorn när utsignalerna blivit stabila. De utlagda signalerna hålls aktiva tills processorn mottaget DTACK. Signalen DTACK genererades av styrlogiken efter att gällande läs- eller skrivcykel var slutförd. Tiden tills det att DTACK genererades berodde på hur lång tid en krets behövde vara aktiv. Processorn klockas med en frekvens på 8 Mhz, om resurserna används på ett effektivt sätt är det fullt tillräckligt för applikationen. Det centrala för att ta vara på processorns resurser är avbrottshantering. Processorn stödde två olika avbrottsnivåer vilket passade konstruktionen väl. De två olika avbrottsnivåerna gavs till DTMT-avkodaren och UART. DTMF-avkodare gavs högst prioritet för att inkommande samtal skall prioriteras framför inkommande data från PCn.

4.2. 16 kb EPROM, Fairchild NM27C128

EPROM användes för lagring av processorns programkod. 16kb valdes för att möjliggöra en utveckling av systemet i framtiden. Det visade sig emellertid att minnet var i största laget.

4.3. 32 kb SRAM, NEC μ PD43256B

SRAM var anpassad till en 8 bitars databuss och 15 bitars adressering. SRAM användes för lagring av data i form av telefonnummer, tid och svartlistning. Listan för lagring av inkomna samtal med tillhörande tid tar 15kb. Läsning och skrivning till SRAM gjordes utan fördröjning, vilket innebar att DTACK genereras samtidigt som

läs- respektive skrivcykeln inleds. Detta var möjligt på grund av att minnet var tillräckligt snabbt och att processorn arbetar synkront, vilket gjorde att detektering av DTACK försköts några klockpulser.

4.4. Realtidsklocka, National Semiconductor MM 58274 C

Realtidsklockan hade en 4 bitars databuss och 4 bitars adressering. Realtidsklockan användes till registrering av tid för inkomna samtal. Klockan hade också en alarmfunktion som kunde programmeras till att ge ”alarm” efter en angiven tid. Denna funktion användes för att styra tiden för hur länge ett svartlistat samtal ska vara inkopplat.

4.5. DTMF-Transceiver, MITEL MT8880C

DTMF-Transceiver valdes för att den både kan avkoda och sända DTMF-signaler. En DTMF-signal detekteras om frekvensen ligger inom bandbredden för avkodning och om amplituden överskrider en satt hysteres. Detekteringen av signaler kan justeras genom att välja lämpliga värden på återkopplingskomponenterna. Det gällde att hitta en nivå där störningar slapp igenom och att DTMF-signaler detekterades. Vid korrekt detektering av en DTMF-signal genererades ett avbrott, vilket låg aktivt tills det att avbrottsregistret lästes. DTMFens utformning gjorde att det blev lämpligt att lägga ut den på två olika adressområden. Adressen 40000 utgjorde funktioner för läsning och sändning av DTMF-signaler. För programmering och initiering av DTMF användes adressen 50000.

4.6. RS-232 Driver, Maxim MAX232

MAX är en analog spänningsomvandlare för UART. MAX inverterade och höjde spänningen för utgående signaler och inverterade och sänkte inkommande signalers spänning. På datorsidan satt det en liknande spänningsomvandlare vilket gjorde att det inte blev någon förvirring gällande inverteringen.

4.7. Kommunikationskrets, UART National PC16550D

UART är en kommunikationskrets som tar in parallell data och sänder ut seriell data. Denna UART valdes för att den hade en 8 bitars databuss. Överföringshastighet och stoppbitar kunde programmeras efter behag och valdes till 19200 bps och 1 stoppbit. Den datoriserade nummerpresentatören krävde stor dataöverföringskapacitet eftersom den skall kunna överföra 15 kb utan dataförlust. För att uppnå detta utnyttjades ett inbyggt ut- och ingångsbuffertsystem på vardera 8 bitar. Buffertsystem lagrade data i en kö och skickade ut det när enheten var ledig. Kön i den ingående bufferten begränsades av att avbrott genererades så fort kön överstigit noll, vilket medförde att köns storlek begränsades maximalt.

4.8. Logikrets, Lattice 1016e

Lattice är en programmerbar logikrets med 32 stycken I/O-pinnar. Denna krets utgjorde all styrlogik för all hårdvara i vår konstruktion. Kretsen genererade CS-signtaler respektive läs- och skrivsigntaler till berörd enhet utifrån processorns styrpinnar. Logiken var stommen i konstruktionen då alla accesser till enheter var direkt beroende av denna. Svårigheten med styrkretsen var att det fanns en tendens till instabilitet hos de utgående signtalerna. Vid tillfällen då många signtaler ändrade värde under samma klockcykel bidrog det till störningar och ibland felaktiga signtaler. Utformningen av Lattice logiken präglades av enkla signtalvägar framför optimering av tid och I/O-pinnar. Konsekvensen blev att nästan all pinnarna på Lattice blev utnyttjade.

4.9. Svartlistning

Svartlistning skall blockera oönskade samtal från att nå den inkopplade telefonen. Det hela fungerade så att ett inkommet samtal avkodas och telefonnumret matchas mot en svartlista för att urskilja om numret är önskvärt eller inte. Om numret ej fanns med skickades telefonsigntalerna vidare ut till den inkopplade telefonen. I det andra fallet då numret var svartlistat adresserade processorn 60000 (koppla in samtal) och kopplade in ett 600 ohms motstånd mellan de ingående telefonlinjerna (600 ohm motsvarar samma resistans som i en telefon med lyft lur). Samtidigt som motståndet kopplades in via ett relä aktiverades realtidsklockans alarmfunktion, som efter 0.1 sekunder kopplade in telefonen igen vilket resulterade i att det inkomna samtalet snabbt kopplades bort. DTMF-signtalerna skickas före ringsigntalen vilket gjorde det möjligt att koppla in och sedan ur samtalet innan någon ringsigntal detekteras.

4.10. Missat samtal

Missat samtal indikeras av en lysande röd diod. När ett inkommande samtal detekteras tänds dioden och blir lysande tills luren lyfts. Nyckeln bakom detektering av missat samtal är strömmätning. Det fanns flera kluriga problemställningar bakom implementeringen. Det går ström i genom kretsen både vid ringsigntaler och samtal. Telefonlinjen har ingen referens till jord och polariteten kan växla helt om beroende på vilket skede i uppkopplingen den är i. Det hela löstes relativt enkelt med en optokopplare parallellt med ett vridmotstånd. Vridmotståndet kalibrerades tills det att optokopplaren endast reagerade på lyft telefonlur.

5. Mjukvara

För att få all hårdvara att fungera tillsammans krävdes en hel del mjukvara. Systemet byggdes upp kring en processor som naturligtvis behövde programmeras. Processorn bestämde vad som skulle göras genom att skriva och läsa till de olika kretsarna. Styrlogiken användes för att generera läs- och skrivcyklar till respektive periferienhet på ett sådant sätt att endast en krets var aktiv åt gången. Tillsist implementerades ett program på PC-sidan som kunde kommunicera med hårdvaran och utgöra gränssnittet mot den verkliga användaren.

5.1. Processor

Det var i programmet för processorn som alla hårdvaronära funktioner implementerades. Huvudprogrammet var skrivet i C med en del initieringssteg skrivna i assembler.

Här följer en översiktlig beskrivning av hur programmet för processorn fungerar:

Vid uppstart av systemet initieras processorns register. Stackpekaren lades vid de högsta minnesadresserna (stacken växer nedåt), programräknare nollställdes, interna pekare och variabler initierades och avbrottshanteringen konfigurerades. Efter det att initieringsprocessen var avklarad allokerade processorn minnesadresser för de olika datastrukturerna. Processorn initierade sedan alla periferienheter genom att skriva till var och en så att kretsarnas konfigurationsregister blev korrekt satta. Därefter lade sig processorn i en evig loop och inväntade att något avbrott skulle inträffa.

Samtalslistan bestod av en 1000*14 byte stor vektor där telefonnummer och tid lagrades. För effektiv minnesanvändning kodades två siffror i varje byte. Ett telefonnummer tog upp 132 bitar jämförbart med en optimal kodning som skulle legat på 93 bitar. Fördelen med att koda på vårt sätt var att algoritmerna blev enkla och tidseffektiva. Svartlistan utgjordes av en 100*8 bytes stor vektor med samma kodning som för samtalslistan. Den skriver till var och en så att alla kontrollregisterna i kretsarna är korrekt satta.

Processorn hade två avbrott, ett från DTMF-kretsen och ett från kommunikationskretsen. När ett avbrott kom från DTMF-kretsen innebar det att en DTMF-signal var detekterad på telefonlinjen. När detta skedde läste processorn från DTMF-kretsen och kontrollerade att det var startbiten för ett nummer som skickades ut. Var så fallet inväntade processorn ett antal siffror följt av koden som angav slutet av numret. När ett helt nummer detekterades, läses tidpunkt in från realtidsklockan vilket tillsammans med numret kodades och lades in i samtalslistan. Därefter skickades numret och datumet till PC:n via kommunikationskretsen. Numret jämfördes också med svartlistan. Förekom numret med i listan, adresserades processorn en speciell adress som indikerade på att logikretsen ska koppla bort samtalet. När ett telefonnummer var detekterat tände även processorn en lysdiod för att indikera att någon har ringt, genom att en signal skickades till logikretsen på liknande sätt som i fallet med svartlistning.

Ett avbrott från kommunikationskretsen innebar antingen att ny data från PC:n var mottagen eller att kretsen var klar för att skicka iväg mer data. När ett avbrott kom lästes först processorn från ett register på kommunikationskretsen som visade på vad som orsakat avbrottet. Berodde avbrottet på att kretsen var klar med att skicka iväg mer data så åtgärdade processorn det, eller markerade internt att ny data kunde skickas iväg när så önskades. Berodde istället avbrottet på att ny data hade inkommit till kommunikationskretsen från PC:n lästes den av processorn och jämfördes med ett antal förutbestämda koder för olika kommandon. PC:n kunde skicka koder för begäran av samtalslista, svartlista och tid. Den kunde även skicka koder för att ladda ner listor och tid till hårdvaran. När processorn detekterade en sådan kod utförde den det som PC:n hade begärt, d.v.s. skickade iväg eller tog emot lista och tid.

Övriga detaljer i processorprogrammet finns att läsa i den bifogade källkoden, se bilaga 2.

5.2. Logik

Logikretsen såg till att endast den krets som processorn adresserade var aktiv. Den såg också till att alla läs- och skrivcyklar genererades på korrekt sätt. De olika kretsarna hade olika specifikationer för hur läs- och skrivcyklar skulle se ut så logikretsen gav olika signaler olika länge beroende på vilken krets det är som gällde. För att processorn skulle avsluta sina läs- och skrivcyklar behövde den en signal som sa att läsningen/skrivningen var klar (DTACK) vilket också logikretsen genererade. När DTMF-kretsen eller kommunikationskretsen gav avbrott så gick dessa via logikretsen in i processorn så att processorn skulle kunna veta vilken krets det var som hade genererat avbrottet.

Programmet som implementerar alla funktioner i logikretsen finns i bilaga 3.

5.3. PC-program

All kommunikation med användaren valde vi att utföra via en PC. För att göra detta smidigt skrev vi ett Windowsprogram i Microsoft Visual C++ som låter användaren kommunicera med hårdvaran.

Programmet tog emot nummer från hårdvaran när någon ringde och presenterade detta. Fanns ett namn associerat med numret, vilket enkelt gjordes inne i programmet, kom detta också upp. Fanns inget namn associerat med numret kunde programmet söka på Eniros telefonkatalog på Internet efter ett namn om så önskades. När ett nummer togs emot sparades det också i en lista vilken sedan gick att spara på hårddisken.

Om inte datorn har varit igång undertiden det har kommit in samtal kunde programmet ladda ner hela samtalslistan från hårdvaran. Vidare hade användaren möjlighet att enkelt definiera en svartlista som programmet sedan skickade över till hårdvaran. Man kunde även ställa tid och datum i hårdvaran via programmet.

6. Resultat och Diskussion

Att gå från idé till fungerande prototyp på sju veckor innebar ett ganska intensivt arbete. Allting tog längre tid än planerat och teorin överensstämde inte alltid med verkligheten. Projektet gav god förståelse över komplexiteten i felsökning. Det kunde ta lång tid efter att ett misstag införts tills att det visade sig i konstruktionen. Svårigheten i felsökandet var att ta med implementering långt tillbaka i tiden i beräkningarna. Enkla saker kunde därför vålla stora problem och leda till omfattande systematiska felsökningar, vilket var tidskrävande. Problem har dykt upp i stort sett alla kretsar, vilket i många fall har berott på dåliga läs- och skrivcyklar. Logikretsen som utgör en vital del av signalkommunikationen var betydligt mer känslig ur störningssynpunkt än beräknat. På grund av att styrkretsen förbisågs som felkälla ledde det till att många kretsar blev felaktigt anlagade. Vanliga problem var att kretsarna fungerat "nästan" eller oregelbundet. Det var sällan svart eller vitt utan mer gråskaligt rätt igenom projektarbetet. DTMF-avkodaren gav till exempel korrekta detekteringar utan att generera något avbrott. Det visade sig till sist att skrivcyklarna under programexekvering inte var tillräckligt stabil. Resultatet vid manuell skrivning och läsning visade inte på några fel. Den största lärdomen av projektet var vikten av stabilitet, noggrann analys och verifiering av signaler för läsning och skrivning. Det är viktigt att mäta tills resultatet är hundra procentigt korrekt innan utvecklingen går vidare, saker sker aldrig av en slump.

Den färdigställda datoriserade nummerpresentatör prototypen var i linje med kravspecifikationen. Kravet om uppringning av telefonnummer från PC blev aldrig implementerat. Mycket tid lades ner på att lösa problemet men utan stabilt resultat. Lagring av nummer har utökats och några funktioner på både på hårdvaru- som mjukvarosidan har lagts till. Den slutgiltiga konstruktionen fungerade bra och kan mycket väl vidareutvecklas till en kommersiell produkt. Produkten skall ses mer som en idébeskrivning än ett vattentätt system för nummerpresentation.

7. Bilagor

7.1. Bilaga 1, Kopplingsschema

7.2. Bilaga 2, Processorprogram

```

/*****
 *
 * Processor program to computerized callerID
 *
 *****/

#define CALL_LIST_SIZE 1000; /* Size of Call List */
#define BLACK_LIST_SIZE 100; /* Size of Black List */

char *addr; /* Global address pointer free to use */

char *comAddr; /* Address to com circuit */
char *clkAddr; /* Address to real time clock */
char *dtmfAddr; /* Address to DTMF transceiver */

/* Variables to maintain com sending */

char comSendReady;
char comSendData[100];
char comSendIndexFirst;
char comSendIndexLast;

char receivingNbr;
int receivingTime;
int receivingBlackList;
int receivingCallList;
char tmp_dtmf;
char year;
char date[14];

/* Variables to maintain Call List and Black List */

char callList[1000][14];
int callListIndexLast;
char callListFull;

char blackList[100][8];
char blackListSize;

/* Initizilation functions */

void comInit(); /* Set up communication circuit */
void clkInit(); /* Set up real time clock */
void dtmfInit(); /* Set up DTMF transceiver */
void redLightInit(); /* Reset red light */

/* Communication functions */

void comHandler(char data); /* Handles communication, decoding commands */
void comSendCall(int index); /* Sends one call to PC */
void comSendCallList(); /* Sends Call List to PC */
void comSendBlackList(); /* Sends Black List to PC */
void comSendTime(); /* Sends date and time to PC */
void comSendAdd(char data); /* Adds a char in out buffer to send to PC */
void comSend(); /* Sends one char from out buffer to PC */

/* Misc functions */

void dtmfHandler(char data); /* Handles DTMF, decoding */
void blackCheck(); /* Checks if number is black listed */
void setTime(); /* Sets time in real clock */
void callListAddTime(int callListIndexLast); /* Adds time to a Call List entry */
void wait(); /* dummy wait */

/* Interrupt functions */

void exp2(); /* Interrupt level 2, used for Com interrupt */
void exp5(); /* Interrupt level 5, used for DTMF interrupt */

```

```

/*****
 * Main program
 *****/

main() {
    _avben(); /* enable interrupt in proc */
    clkAddr = (char*) 0x20000;
    comAddr = (char*) 0x30000;
    dtmfAddr = (char*) 0x40000;
    callListIndexLast = 0;
    callListFull = 0;

    receivingNbr = 0;
    receivingTime = 0;
    comSendReady = 1;
    comSendIndexFirst = 0;
    comSendIndexLast = 0;
    year = 0;

    receivingBlackList = 0;
    receivingCallList = 0;

    blackListSize = 0;

    wait();
    comInit();
    wait();
    clkInit();
    wait();
    dtmfInit();
    wait();
    redLightInit();
    wait();

    greta: goto greta; /* loop forever */
}

void comInit() { /* Set up communication circuit */

    /* Set baud rate to 19200 bps */
    addr = (char*) 0x30003; /* Line Control Register */
    *addr = 0x80; /* DLAB=1 */

    addr = (char*) 0x30000; /* Divisor Latch (LS) */
    *addr = 0xB; /* 0x15 */

    addr = (char*) 0x30001; /* Divisor Latch (MS) */
    *addr = 0;

    /* Set Line Control */
    addr = (char*) 0x30003; /* Line Control Register */
    *addr = 0x3; /* 8 bits word length, 1 stop bit, no parity check, DLAB=0. */

    /* Enable FIFO-mode */
    addr = (char*) 0x30002; /* FIFO Control Register */
    *addr = 0; /* 7 */

    /* Set Interrupt */
    addr = (char*) 0x30001; /* Interrupt Enable Register */
    *addr = 0x7; /* Enable interrupt when: */
    /* * Received Data Available */
    /* * Transmitter Holding Register Empty */
    /* * Receiver Line Status */

    addr = (char*) 0x30002; /* FIFO Control Register */
    *addr; /* Clear old interrupts */

    addr = (char*) 0x30000; /* Receiver Buffer Register */
    *addr; /* Clear old data */
}

void clkInit() { /* Set up real time clock */

    int i;

    addr = (char*) 0x20000; /* Control Register */
    *addr = 0xF; /* Test mode, Clock halted, Interrupt Register enable */

    addr = (char*) 0x2000F; /* Interrupt Register */
    *addr = 0; /* Disable interrupts */
}

```

```

    addr = (char*) 0x20000; /* Control Register */
    *addr = 5; /* Normal mode, Clock halted, Time Setting Register enable */

    /* Set all registers to valid BCD values */
    for (i = 1; i < 0xF; i++) {
        addr = clkAddr + i;
        *addr = 1;
    }

    addr = (char*) 0x2000F; /* Time Setting Register */
    *addr = 1; /* Set 24-hours mode */

    /* Set Time and Date */
    addr = (char*) 0x20004; /* Unit Minutes */
    *addr = 5;

    addr = (char*) 0x20005; /* Tens Minutes */
    *addr = 0;

    addr = (char*) 0x20006; /* Unit Hours */
    *addr = 9;

    addr = (char*) 0x20007; /* Tens Hours */
    *addr = 1;

    addr = (char*) 0x20008; /* Unit Days */
    *addr = 9;

    addr = (char*) 0x20009; /* Tens Days */
    *addr = 0;

    addr = (char*) 0x2000A; /* Unit Months */
    *addr = 2;

    addr = (char*) 0x2000B; /* Tens Months */
    *addr = 0;

    addr = (char*) 0x2000C; /* Unit Years */
    *addr = 4;

    addr = (char*) 0x2000D; /* Tens Years */
    *addr = 0;

    addr = (char*) 0x2000E; /* Day of Week */
    *addr = 1;

    /* Set interrupt */
    addr = (char*) 0x20000; /* Control Register */
    *addr = 3; /* Enable interrupt register, interrupt stopped */
    addr = (char*) 0x2000F; /* Interrupt Register */
    *addr = 1; /* Enable interrupt 0.1 second */

    /* Start clock */
    addr = (char*) 0x20000; /* Control Register */
    *addr = 1; /* Normal mode, Clock started */
}

void dtmfInit() { /* Set up dtmf transceiver */
    char tmp;

    addr = (char*) 0x50000; /* Control Register */

    tmp = *addr; /* init stuff */
    *addr = 0; /* init stuff */
    *addr = 0; /* init stuff */
    *addr = 8; /* init stuff */
    *addr = 0; /* init stuff */
    tmp = *addr; /* init stuff */

    *addr = 0xD; /* Enable interrupt, tone output */
    *addr = 0; /* 1; /* No burst mode, no test mode */
}

void redLightInit() {
    int i;
    addr = (char*) 0x80000;
    *addr = 0;

```



```

    for (i=0;i<100;i++)
        wait();
    addr = (char*) 0x70000;
    *addr = 0;
    for (i=0;i<500;i++)
        wait();

    addr = (char*) 0x80000;
    *addr = 0;
}

void comHandler(char data) {
    if (data == 0x1D){
        comSendCallList();
    }else if (data == 0x2E){
        comSendTime();
    }else if (data == 0x3B) { /* receive time*/
        *clkAddr = 5; /* Normal mode, Clock halted, Time Setting Register enable */
        receivingTime = 1;
    } else if (data == 0x2B) { /* Send BlackList to computer*/
        comSendBlackList();
    } else if (data == 0x2C) { /* Start receiving BlackList from computer*/
        receivingBlackList = 1;
        blackListSize = 0;
    }else if (data == 0x2D) { /* Stop receiving BlackList*/
        receivingBlackList = 0;
    }else if (receivingBlackList){
        blackList[blackListSize][receivingBlackList-1]=data;
        if ( ((data & 0xF) ==0xA) || ((data & 0xF0) ==0xA0)){
            receivingBlackList=1;
            blackListSize++;
        } else{
            receivingBlackList++;
        }
    } else if (data == 0x1B){
        comSendAdd(0x1B);
    } else if (data == 0x1E){ /* start receiving Call List */
        receivingCallList = 1;
        callListIndexLast = 0;
        callListFull = 0;
    } else if (data == 0x1F){ /* stop receiving Call List */
        receivingCallList = 0;
    } else if (receivingCallList){
        callList[callListIndexLast][receivingCallList-1]=data;
        if ( ((data & 0xF) ==0xA) || ((data & 0xF0) ==0xA0)){
            receivingCallList=1;
            callListIndexLast++;
        } else{
            receivingCallList++;
        }
    } else {
        if (receivingTime) {
            date[receivingTime - 1] = data;
            if (receivingTime >= 0xC) {
                setTime();
                *clkAddr = 0; /* Start clock */
                receivingTime = 0;
            }
            receivingTime++;
        }
    }
}

void comSendCall(int index) {
    int i = -1;

    comSendAdd(0x1C); /* Send Call start byte */
    do {
        i++;
        comSendAdd(callList[index][i]);
    } while (((callList[index][i] & 0xF) != 0xA) &&
        ((callList[index][i] & 0xF0) != 0xA0));
}

```

```

}

void comSendCallList() {
    int i;

    comSendAdd(0x1E); /* Send List start byte */

    for (i = 0; i < 1000*callListFull+callListIndexLast*(1-callListFull); i++) {
        comSendCall(i);
    }

    comSendAdd(0x1F); /* Send List stop byte */
}

void comSendBlackList(){
    int i;
    int b=-1;
    comSendAdd(0x2c); /* start byte sending blackList */
    for (i=0;i<blackListSize;i++){
        do{
            b++;
            comSendAdd(blackList[i][b]);
        }while ((blackList[i][b] & 0xF) !=0xA && (blackList[i][b] & 0xF0)!=0xA0 && b<8 );
        b=-1;
    }

    comSendAdd(0x2D); /* stop byte sending black List */
}

void comSendTime() {
    int i;
    for (i = 0xD; i >= 2; i--) {
        addr = (char*) clkAddr + i;
        comSendAdd('0' + (*addr & 0xF));
    }
}

void comSendAdd(char data) {
    comSendData[comSendIndexLast] = data;
    comSendIndexLast = (comSendIndexLast + 1) % 100;

    if (comSendReady)
        comSend();
}

void comSend() {
    if (comSendIndexFirst != comSendIndexLast) {
        comSendReady = 0;
        *comAddr = comSendData[comSendIndexFirst];
        comSendIndexFirst = (comSendIndexFirst + 1) % 100;
    }
}

void dtmfHandler(char data) { /* Takes care of incoming dtmf */

    if (data == 0 || data == 0xC) { /* number begin byte */

        receivingNbr = 13;
        /* Red light ON */
        /*comSendAdd('?');*/
        addr=(char*) 0x70000;
        *addr=0;

    } else if (data == 0xE) { /* number begin byte Info Code*/

        /* Red light ON */

        addr=(char*) 0x70000;
        *addr=0;

        receivingNbr = 13;
    } else if (receivingNbr && data == 0xF) { /* number end byte */

        if (receivingNbr%2)
            callList[callListIndexLast][(receivingNbr-1)/2] = 0xA;
        else
            callList[callListIndexLast][(receivingNbr-1)/2] += 0xA0;
    }
}

```

```

        blackCheck();

        callListAddTime(callListIndexLast);
        comSendCall(callListIndexLast);

        receivingNbr = 0;
        callListIndexLast = (callListIndexLast + 1) % 1000;
        if (callListIndexLast == 0)
            callListFull = 1;

    } else if (receivingNbr) {
        if (data == 0xA)
            data = 0;
        if (receivingNbr % 2)
            callList[callListIndexLast][(receivingNbr-1)/2]=data;
        else
            callList[callListIndexLast][(receivingNbr-1)/2]+=data<<4;
        receivingNbr++;
    }
}

void blackCheck() {
    int i;
    int b;
    for (b=0;b<blackListSize;b++){
        i=6; /* number comes after time and date*/
        while(i<15){
            if ( (callList[callListIndexLast][i] & 0xF) == (blackList[b][i-6] & 0xF)){
                if ( (blackList[b][i-6] & 0xF) == 0xA){
                    addr=(char*) 0x20000;
                    i = *addr; /* reset interrupt */
                    addr=(char*) 0x60000;
                    *addr=0; /* block call */
                    addr=(char*) 0x20000;
                    *addr = 0; /* start time-interrupt */

                    *addr=0; /* enable interrupt*/
                    i=15;
                }
            }else{
                i=15;
            }
            if ( (callList[callListIndexLast][i] & 0xF0) == (blackList[b][i-6] & 0xF0)){
                if ( (blackList[b][i-6] & 0xF0) == 0xA0){
                    addr=(char*) 0x20000;
                    i = *addr; /* reset interrupt */
                    addr=(char*) 0x60000;
                    *addr=0; /* block call */
                    addr=(char*) 0x20000;
                    *addr = 0; /* start time-interrupt */
                    i=15;
                }
            }else{
                i=15;
            }
        }
        i++;
    }
}

void setTime() {
    int i;

    addr = (char*) 0x2000F;
    *addr = 1 + 4 * ((date[0] - '0' + date[1] - '0') % 4); /* set leap year */
    for (i = 0; i < 12; i++) {
        addr = (char*) clkAddr + 0xD - i;
        if (date[i] >= '0') /* Safety test */
            *addr = date[i] - '0';
    }
}

void callListAddTime(int callListIndexLast) {
    int i;

    for (i = 0; i < 12; i++) {
        addr = (char*) 0x2000D - i;
        if (i % 2)
            callList[callListIndexLast][i/2] += (*addr & 0xF) << 4;
        else
            callList[callListIndexLast][i/2] = *addr & 0xF;
    }
}

```

```
    }  
}  
  
void wait() {  
    int i;  
    for (i = 0; i < 0x55; i++) {  
    }  
}  
  
void exp2() { /* Com interrupt */  
    char status;  
    char data;  
    char id;  
    addr = (char*) 0x30002; /* Interrupt Identification Register */  
    id = *addr;  
  
    if ((id & 0xE) == 6) { /* Receiver Line Status */  
        addr = (char*) 0x30005;  
        status = *addr;  
        if ((status & 2) == 1) { /* Overrun Error */  
        }  
    }  
  
    else if ((id & 0xE) == 0xC || (id & 0xE) == 4) { /* Atleast one character in FIFO */  
        data = *comAddr;  
        comHandler(data);  
    }  
  
    else if ((id & 0xE) == 2) { /* Transmitter Holding Register Empty */  
        comSendReady = 1;  
        comSend();  
    }  
    return;  
}  
  
void exp5() { /* Dtmf interrupt */  
    char id;  
    char data;  
  
    addr = (char*) 0x50000; /* Status Register */  
    id = *addr;  
  
    if ((id & 7) == 5) { /* Valid data */  
        addr = (char*) 0x40000;  
        data = *addr & 0xF; /*Only look at 4 lowest bits */  
        dtmfHandler(data);  
    }  
  
    return;  
}
```

7.3. Bilaga 3, Logikprogram

```

MODULE logic
TITLE 'logic'

"INPUTS"
ADDR16    pin 15;
ADDR17    pin 16;
ADDR18    pin 17;
ADDR19    pin 18;
INTRPT_DTMF pin 19;
INTRPT_COM pin 20;
FC0_PROC  pin 21;
FC1_PROC  pin 22;
FC2_PROC  pin 25;
AS_PROC   pin 26;
DS_PROC   pin 27;
RW_PROC   pin 28;
CLK       pin 11;
REAL_INT  pin 30;
OFF_HOOK  pin 31;

"OUTPUTS"
CS_DTMF    pin 10;
CS_REAL_CLK pin 9;
CS_COM     pin 8;
CS_SRAM    pin 7;
CS_EEPROM  pin 6;
READ_OE    pin 5;
WRITE      pin 4;
IPL02_PROC pin 3;
IPL1_PROC  pin 44;
VPA_PROC   pin 43;
RS0_DTMF   pin 42;
DTACK_PROC pin 41;
FI2_DTMF   pin 39;
READ_WRITE_DTMF pin 37;
BLACK_DTMF pin 38;
RED_LIGHT  pin 40;

STATE1     node ISTATE 'reg';
STATE2     node ISTATE 'reg';
STATE3     node ISTATE 'reg';
STATE4     node ISTATE 'reg';

B_STATE    node ISTATE 'reg';
L_STATE    node ISTATE 'reg';

"STATES"
state_dtmf = [STATE1, STATE2, STATE3, STATE4];
state_black = [B_STATE];
state_light = [L_STATE];
A = 0; B = 1; C = 2; D = 3; E = 4; F = 5; G = 6; H = 7;
I = 8; J = 9; K = 10; L = 11; M = 12; N = 13; O = 14; P = 15;

EQUATIONS
CS_REAL_CLK = (!ADDR19 & !ADDR18 & ADDR17 & !ADDR16 & !AS_PROC & (RW_PROC #
!RW_PROC & !DS_PROC));
CS_COM = (!ADDR19 & !ADDR18 & ADDR17 & ADDR16 & !AS_PROC & (RW_PROC # !RW_PROC &
!DS_PROC));
CS_SRAM = (!ADDR19 & !ADDR18 & !ADDR17 & ADDR16 & !AS_PROC & (RW_PROC # !RW_PROC &
!DS_PROC));
CS_EEPROM = (!ADDR19 & !ADDR18 & !ADDR17 & !ADDR16 & !AS_PROC & (RW_PROC # !RW_PROC
& !DS_PROC));
IPL1_PROC = !INTRPT_COM # !INTRPT_DTMF;
IPL02_PROC = INTRPT_DTMF;
VPA_PROC = !(FC0_PROC & FC1_PROC & FC2_PROC);
RS0_DTMF = !ADDR19 & ADDR18 & !ADDR17 & ADDR16 & !AS_PROC;

[STATE1, STATE2, STATE3, STATE4, B_STATE, L_STATE].clk = CLK;

```

```

STATE_DIAGRAM state_dtrack;

state A:
    DTACK_PROC = CS_SRAM & CS_EPROM & !(ADDR19 & ADDR18 & ADDR17 & !ADDR16 &
!AS_PROC) & !(ADDR19 & ADDR18 & ADDR17 & ADDR16 & !AS_PROC) & !(ADDR19 & !ADDR18 &
!ADDR17 & !ADDR16 & !AS_PROC);
    WRITE = DS_PROC # RW_PROC # AS_PROC;
    READ_OE = !RW_PROC # AS_PROC;
    FI2_DTMF=0;
    CS_DTMF=1;
    READ_WRITE_DTMF = RW_PROC;

    IF (!ADDR19 & ADDR18 & !ADDR17 & !AS_PROC & (RW_PROC # (!RW_PROC & !DS_PROC)) )
THEN F; //READ-WRITE DTMF
    ELSE IF ((!CS_REAL_CLK # !CS_COM) & (RW_PROC # (!RW_PROC & !DS_PROC ))) THEN B;
//COM AND REAL_CLK
    ELSE A;

state B:
    DTACK_PROC = 1;
    CS_DTMF=1;
    FI2_DTMF=0;
    READ_OE = !RW_PROC;
    WRITE = RW_PROC;
    GOTO C;

state C:
    DTACK_PROC = 1;
    CS_DTMF=1;
    FI2_DTMF=0;
    READ_OE = !RW_PROC;
    WRITE = RW_PROC;
    goto D;
state D:
    DTACK_PROC = 1;
    CS_DTMF=1;
    FI2_DTMF=0;
    READ_OE = !RW_PROC;
    WRITE = RW_PROC;
    goto P;

STATE P:
    DTACK_PROC = 1;
    CS_DTMF=1;
    FI2_DTMF=0;
    WRITE = 1;
    READ_OE= 1;
    goto E;

state E:
    DTACK_PROC = 0;
    CS_DTMF=1;
    FI2_DTMF=0;
    WRITE = 1;
    READ_OE= 1;
    IF (AS_PROC) THEN A;
    ELSE E;
STATE F:
    DTACK_PROC = 1;
    FI2_DTMF=0;
    CS_DTMF=0;
    READ_WRITE_DTMF=RW_PROC;
    goto G;

STATE G:
    DTACK_PROC = 1;
    FI2_DTMF=0;
    CS_DTMF=0;
    READ_WRITE_DTMF=RW_PROC;
    goto H;

STATE H:
    DTACK_PROC = 1;
    FI2_DTMF=1;
    CS_DTMF=0;
    READ_WRITE_DTMF=RW_PROC;
    goto I;
STATE I:
    DTACK_PROC = 1;
    FI2_DTMF=1;
    CS_DTMF=0;
    READ_WRITE_DTMF=RW_PROC;
    goto J;
STATE J:

```

```

        DTACK_PROC = 1;
        FI2_DTMF=1;
        CS_DTMF=0;
        READ_WRITE_DTMF=RW_PROC;
        goto K;

STATE K:
        DTACK_PROC = 1;
        FI2_DTMF=0;
        CS_DTMF=0;
        READ_WRITE_DTMF=RW_PROC;
        goto L;

STATE L:
        DTACK_PROC = 1;
        FI2_DTMF=0;
        CS_DTMF=0;
        READ_WRITE_DTMF=RW_PROC;
        goto M;

STATE M:
        DTACK_PROC = 1;
        FI2_DTMF=0;
        CS_DTMF=0;
        READ_WRITE_DTMF=!RW_PROC;
        goto N;

STATE N:
        DTACK_PROC = 1;
        FI2_DTMF=0;
        CS_DTMF=0;
        READ_WRITE_DTMF=!RW_PROC;
        goto 0;

STATE 0:
        DTACK_PROC = 0;
        FI2_DTMF=0;
        CS_DTMF=1;
        READ_WRITE_DTMF=!RW_PROC;
        if (AS_PROC) then A;
        ELSE 0;

STATE_DIAGRAM state_black;

state A:
        BLACK_DTMF = !ADDR19 & ADDR18 & ADDR17 & !ADDR16 & !AS_PROC;
        IF (BLACK_DTMF) THEN B;
        ELSE A;
state B:
        BLACK_DTMF = 1;
        IF (!REAL_INT # (ADDR19 & !ADDR18 & !ADDR17 & !ADDR16 & !AS_PROC)) THEN A;
        ELSE B;

STATE_DIAGRAM state_light;

state A:
        RED_LIGHT = !ADDR19 & ADDR18 & ADDR17 & ADDR16 & !AS_PROC;
        IF (RED_LIGHT) THEN B;
        ELSE A;
state B:
        RED_LIGHT = 1;
        IF (OFF_HOOK # (ADDR19 & !ADDR18 & !ADDR17 & !ADDR16 & !AS_PROC)) THEN A;
        ELSE B;

END

```